

Class 3: Basic Python Programming

Data-driven Systems Engineering

Agenda

- Variables and Python's dynamic typing.
- Numbers, strings, and booleans.
- Lists as Python's basic container type.
- Modules, packages, and libraries: what the words actually mean.
- The Python Standard Library and built-in functions.
- Importing and using an external library.

Learning goals

- Write and reason about variable assignment in a dynamically typed language.
- Distinguish int, float, complex, string, and boolean values.
- Build and modify a list using built-in methods.
- Explain the difference between a module, a package, and a library.
- Import a library and call a function from it with the correct syntax.

Course Map and Running Cases



Today in the course

Focus: The smallest building blocks of Python: variables, types, and lists.

Bridge: Every later notebook, script, and dashboard in this course is built from exactly these pieces, repeated at scale.

Fraud case

The fraud dashboard stores a transaction amount in a variable, a list of behavioral features in a list, and a model accuracy in a float — three lines of code, three of today's topics.

Pasteurization case

The pasteurization simulator stores sensor readings the same way: plain variables and lists before anything is called a “feature” or a “time series.”

Course thread: Systems look complicated from the outside, but the vocabulary underneath is always variables, types, and containers.

Variables and dynamic typing

```
MODEL_ACCURACY = 0.9422  
transaction_amount = 120.0  
page_title = "Credit Card Fraud Detection"
```

- Variables are created with the assignment operator =; there is no separate type declaration.
- Python is **dynamically typed**: a variable's type is decided by the value assigned to it, and can change later.
- These three lines come straight from 'streamlit/pages/App_Credit_Card.py'.

Rules for naming a variable

- Must start with a letter or an underscore, never with a digit.
- May contain only letters, digits, and underscores afterwards.
- Names are case-sensitive: `amount` and `Amount` are different variables.
- Conventionally lowercase with underscores for ordinary variables, uppercase for constants (as in `MODEL_ACCURACY` above).

Arithmetic operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division (always returns a float)
**	Power
//	Integer (floor) division

Numeric types

- **int**: whole numbers, such as a count of transactions or students in a group.
- **float**: decimal numbers, such as `MODEL_ACCURACY = 0.9422` or a sensor reading.
- **complex**: numbers with a real and imaginary part; rarely needed in this course, but part of the standard set.
- Python decides which of these a literal is the moment it is written: 10 is an int, 10.0 is a float.

Strings

```
st.title("Credit Card Fraud Detection System")  
message = "Adjust the sliders to simulate a transaction."  
label = "Feature V" + str(1)
```

- Strings can be written with single or double quotation marks; this course mostly uses double quotes.
- Strings behave like arrays of characters and can be indexed, e.g. `label[0]`.
- Concatenation uses `+`, but only between strings — numbers must be converted with `str()` first.

- Python's boolean type has exactly two values: True and False.
- Comparisons such as `prediction == 1` or `probability > 0.5` evaluate to a boolean.
- Boolean logic uses English keywords rather than symbols: `and`, `or`, `not`.
- Booleans decide almost every branch of code we will write from next class onward.

Lists

```
user_inputs = []  
user_inputs.append(0.0)  
  
input_data = [transaction_amount] + user_inputs  
print(len(input_data))
```

- A list is Python's ordered, resizable container; elements do not need to share a type.
- `.append()` is a built-in list method; `+` between two lists concatenates them.
- 'App_Credit_Card.py' builds exactly this kind of list to collect the 28 behavioral features of a transaction.

Modules, packages, and libraries

- A **module** is one file of Python code: functions, classes, and variables collected together.
- A **package** is a directory of modules, marked so Python can import from it.
- A **library** is a collection of modules and packages meant to be reused across projects.
- The Python Standard Library ships with the interpreter: file I/O, basic mathematics, and much more, with no installation needed.

A first look at the Standard Library

```
import math

math.sqrt(16)      # 4.0
math.log10(1000)   # 3.0
math.pi            # 3.14159...
```

- The `math` module ships with every Python install: trigonometric, logarithmic, and constant values.
- `statistics`, another Standard Library module, adds `mean()` and `stdev()` without installing anything.

A handful of built-in functions

Function	What it does
<code>len()</code>	Number of elements in a list, string, or other container.
<code>type()</code>	The type of a value, useful while debugging dynamic typing.
<code>print()</code>	Writes to standard output; the most-used function in this course.
<code>sum()</code> / <code>sorted()</code>	Aggregate or order the elements of a list.
<code>range()</code>	Generates a sequence of integers, mostly seen together with loops.

Importing an external library

```
import streamlit as st
import numpy as np
import pandas as pd
```

- `import x as y` imports the whole library under a short alias — these three aliases are used everywhere in this repository.
- Standard Library modules (`math`, `statistics`) never need installation; external libraries (`streamlit`, `numpy`, `pandas`) do, which is exactly what `'requirements.txt'` is for.

How the repo already uses today's building blocks

Repo activity

- 'streamlit/pages/App_Credit_Card.py' opens with plain variables (MODEL_ACCURACY), a list (user_inputs), and library imports before anything ML-specific happens.
- 'streamlit/modeling.py' stores a dataset URL in a string variable and a target column name in another — the exact pattern from today's slides.
- Skim either file and count how many lines are only variables, types, or imports before the first model-specific call.

Question

If a variable's type can change at any time in Python, what could go wrong in a long script, and how would you protect against it?

- Surfaces the trade-off behind dynamic typing before students hit a confusing type error later in the course.
- Motivates disciplined naming and, eventually, type hints.

Papers and materials

[Jupyter install guide](#); [pandas documentation](#); Chip Huyen, *Designing Machine Learning Systems*; Noah Gift and Alfredo Deza, *Practical MLOps*.

From This Class to the Next

What stays with us

- Variables, numeric and string types, booleans, and lists are the vocabulary every later script reuses.
- Modules, packages, and libraries describe how Python code is organized and shared, not just where files live.
- Reading real repository files is already possible once these basics are solid.

Project use

Before writing project code, make sure you can read a file like 'App_Credit_Card.py' line by line and name what each line does.

Next connection

Class 4 moves into control flow — conditionals, loops, and dictionaries — so these values can start driving decisions rather than sitting still.

Course thread: Today's static values become tomorrow's dynamic behavior.